

AI SMART EXPENSE TRACKER

& SAVINGS PLANNER

Developer Documentation & Technical Playbook

This comprehensive playbook combines architectural designs, schema descriptions, run manuals, and frontend/backend code explanations for the AI-powered smart budgeting ecosystem.

PROJECT ARTIFACT & MANUALS

Compiled Automatically | Target Build: v1.0.0

Environment Stack: Python Flask + React.js + MongoDB

Project Guide: AI Smart Expense Tracker & Savings Planner

This guide serves as a comprehensive developer handbook and execution playbook for the **AI Smart Expense Tracker & Savings Planner** application. It details the system architecture, file directory map, environment configurations, core code mechanics, and staging deployment steps.

[Workspace Map] Workspace Directory Layout

The codebase is organized into two isolated modules: a backend Flask service and a frontend React single-page application.

```

c:\Users\md313\OneDrive\Desktop\ai\
??? README.md                (Master Setup Documentation)
??? backend/
?   ??? .env                  (Local configuration properties)
?   ??? requirements.txt      (Backend Python dependencies)
?   ??? app/
?       ??? app.py            (Flask core server entrypoint)
?       ??? config/
?           ? ??? config.py    (Loads environment variables)
?       ??? routes/
?           ? ??? auth.py      (User registration & JWT login APIs)
?           ? ??? expenses.py  (Transactions log ledger CRUD APIs)
?           ? ??? savings.py   (Milestone savings planner CRUD APIs)
?           ? ??? ai.py        (OpenAI Insights & rule-based engine APIs)
?       ??? utils/
?           ??? db.py          (MongoDB PyMongo database helper)
?           ??? auth_middleware.py (Token check decorator helper)
??? frontend/
    ??? .env                  (Vite frontend configurations)
    ??? package.json          (Frontend node package list)
    ??? src/
        ??? index.css         (Nebula glassmorphic styles system)
        ??? App.jsx           (Router page frames layout)
        ??? main.jsx          (React root wrapper)
        ??? api/
            ??? index.js       (Axios base client & endpoint mappings)
        ??? context/
            ??? AuthContext.jsx (Active session and token states)
        ??? components/
            ??? ProtectedRoute.jsx (Navigation guard loader)
            ??? Sidebar.jsx      (Collapsible menu with active states)
            ??? Navbar.jsx       (Toolbars, calendar, notifications, currency selector)
        ??? pages/
            ??? Login.jsx       (Credentials portal)
            ??? Register.jsx     (Account enrollment form)
            ??? Dashboard.jsx    (KPI overview widgets & Chart.js panels)
            ??? Expenses.jsx     (Ledger manager filters & logging drawers)
            ??? Savings.jsx      (Milestone boards & calculators)
            ??? AIInsights.jsx   (Projections chart & PDF printable builder)

```

[Configuration] Configuration & Variables

Backend Environment Configuration (`backend/.env`)

Ensure these properties match your database parameters:

```
# Database URI: Local connection fallback or MongoDB Atlas string
MONGO_URI=mongodb://localhost:27017/expense_tracker

# Cryptographic token signing secret key (keep secure in production!)
JWT_SECRET=ai_smart_expense_tracker_secret_998877

# Optional OpenAI API Key (Leave blank to use the offline rules statistics model)
OPENAI_API_KEY=your-openai-api-key-here

# Server properties
PORT=5000
DEBUG=True
```

Frontend Environment Configuration (`frontend/.env`)

Defines the base route endpoint for outgoing Axios HTTP calls:

```
VITE_API_URL=http://localhost:5000/api
```

[Setup] Local Run Playbook

Follow these sequential instructions to run the application locally.

Phase 1: MongoDB Database Setup

Ensure that MongoDB is running on your machine:

- **Windows:** Press `Win + R`, type `services.msc`, locate `MongoDB Server`, and click **Start**.
- **Mac:** Run `brew services start mongodb-community`.
- **(Alternatively)** Set a MongoDB Atlas Connection SRV link in `backend/.env`.

Phase 2: Start Flask Backend Server

1. Open your terminal and enter the `backend/` folder:

```
cd backend
```

2. Set up and activate a standard virtual environment:

```
# Windows PowerShell
python -m venv venv
.\venv\Scripts\Activate.ps1
```

```
# Mac / Linux Bash
python3 -m venv venv
source venv/bin/activate
```

3. Install Python modules:

```
pip install -r requirements.txt
```

4. Launch Flask:

```
python app/app.py
```

Phase 3: Start Vite React Frontend

1. Open a new terminal window and navigate to the `frontend/` folder:

```
cd frontend
```

2. Install dependencies:

```
npm install
```

3. Launch the Vite hot-reloading dev server:

```
npm run dev
```

4. Navigate to `http://localhost:5173` in your web browser.

[Code Mechanics] Core Code Mechanics

1. Salting Credentials & Secure Auth Flow

Registration cryptographically hashes passwords using a secure salting configuration in `backend/app/routes/auth.py`:

```
salt = bcrypt.gensalt(12)
password_hash = bcrypt.hashpw(password.encode('utf-8'), salt)
```

Upon verification, the backend generates a secure JSON Web Token signature:

```
token_payload = {
    "user_id": str(user['_id']),
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=24)
}
token = jwt.encode(token_payload, Config.JWT_SECRET, algorithm="HS256")
```

This token is passed back, saved in `localStorage`, and injected by the Axios request interceptor (`frontend/src/api/index.js`) in standard Bearer headers for protected pages.

2. Live Currency System Event Dispatching

To instantly convert currency values across every active dashboard component without page refreshes, `Navbar.jsx` dispatches a custom window event whenever the selector is toggled:

```
localStorage.setItem('app_currency', selected);
window.dispatchEvent(new Event('currencyChange'));
```

Other dashboard widgets register a listener on load to immediately update their labels in real-time:

```
useEffect(() => {
  const onCurrencyChange = () => {
    setCurrency(localStorage.getItem('app_currency') || 'Rs. ');
  };
  window.addEventListener('currencyChange', onCurrencyChange);
  return () => window.removeEventListener('currencyChange', onCurrencyChange);
}, []);
```

3. Dynamic Savings Milestones & Progress Math

Savings goals compute timeline values dynamically in `backend/app/routes/savings.py`:

- **Progress Ratio:** `round((saved_amount / target_amount) * 100, 2)` (capped between 0 and 100).
- **Time Horizon Remaining:** Average number of months remaining derived from active deadline dates:

```
delta = goal["deadline"] - today
days = delta.days
months_remaining = max(1.0, round(days / 30.44, 1))
```

- **Velocity Needed:** Dynamic monthly contribution rate to secure targets on time: `(target_amount - saved_amount) / months_remaining`.

4. Robust Dual-Engine AI Fallback Pipeline

In `backend/app/routes/ai.py`, the analysis route checks if a standard OpenAI key is configured. If available, it queries GPT-3.5 using structured JSON parameters:

```
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[...],
    response_format={"type": "json_object"}
)
```

If the API key is omitted, it falls back to the **Local Statistical Engine**. This engine parses the database aggregates to compile real overspending ratios and actionable budget rules automatically:

- Identifies if discretionary categories (e.g. Shopping or Entertainment) exceed 35% of overall outflow.
- Triggers reminders on time-sensitive milestones.
- Proposes mathematical spending reductions based on current transaction frequency velocities.

5. Print-Optimized PDF Ledger Reports

To offer printable ledger reports without heavy PDF package sizes, `AllInsights.jsx` builds a print-optimized stylesheet window, populates it with transactions and savings tables, and runs a print request:

```
const printWindow = window.open('', '_blank');
printWindow.document.write(compiledReportHTML);
printWindow.document.close();
// Inside compiled HTML: window.onload = () => window.print();
```

This cleanly invokes the OS native **Print to PDF** systems, making it highly compatible across mobile devices and desktops.

[Production Deployment] Production Deployment Blueprint

Database: MongoDB Atlas

1. Register at [MongoDB Atlas](https://www.mongodb.com/cloud/atlas).
2. Build a free M0 Sandbox cluster.
3. Under Database Access, create a database user and record credentials.
4. Under Network Access, allow access from anywhere (`0.0.0.0/0` - standard for Render hosting).
5. Click Connect, choose "Drivers", and copy your `mongodb+srv://...` SRV connection link.

Backend: Render

1. Push your code repository to GitHub.
2. Login to [Render](https://render.com/) and click **New > Web Service**.
3. Connect your GitHub repository.
4. Specify these variables:
 - **Environment:** `Python`
 - **Build Command:** `pip install -r backend/requirements.txt`
 - **Start Command:** `gunicorn --cwd backend/app app:app` (ensure `gunicorn` is installed or run `python backend/app/app.py`).
5. In the Environment panel, add your environment variables:
 - `MONGO\_URI` = *Your MongoDB Atlas connection link*
 - `JWT\_SECRET` = *Secure random string*
 - `OPENAI\_API\_KEY` = *Your OpenAI Key*
 - `PORT` = `10000`

Frontend: Vercel

1. Install the Vercel CLI or login to the [Vercel Dashboard](https://vercel.com/).
2. Click **Add New > Project**, and import your GitHub repository.
3. Configure these framework parameters:
 - **Framework Preset:** `Vite`
 - **Root Directory:** `frontend`
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
4. Add environment variables:
 - `VITE\_API\_URL` = *Your deployed Render backend URL (e.g. https://your-backend.onrender.com/api)*
5. Click **Deploy**. Vercel will build and assign a public SSL URL for your single-page app!

README & User Quickstart

AI Smart Expense Tracker & Savings Planner

A gorgeous, premium, dark-mode glassmorphic full-stack application for modern asset tracking, savings planning, and AI-driven budget coaching. Built using **Flask (Python)**, **React.js (Vite)**, **MongoDB**, and **OpenAI GPT-3.5 API**.

[Key Features] Key Features

- **Premium Glassmorphic Interface:** Sleek glowing dark theme, interactive cards, smooth page transitions, responsive layouts, and animations.
- **Secure Authentication:** Secure sign-up/logins using robust password hashing (`bcrypt`) and protected pages secured by JSON Web Tokens (`jwt`).
- **Ledger Transaction Hub:** Advanced expense logger supporting categorization (Food, Travel, Shopping, Bills, Entertainment, Health, Education), date selection, search indexing, and range filters.
- **Savings Milestone Modeler:** Interactive milestone boards featuring completion percentages, target deadlines, months remaining trackers, and suggested monthly velocities.
- **Interactive Calculator Widget:** Model exact savings time horizons based on customizable monthly contributions.
- **Dual AI Financial Coach:**
 - **OpenAI GPT-3.5:** Full natural language analysis, overspending warning logs, and dynamic budget action advice.
 - **Rule-Based Fallback Engine:** Mathematical statistics engine that triggers instantly if `OPENAI\_API\_KEY` is omitted, guaranteeing fully-functional offline modeling.
- **Analytical Projections:** Forecast next month's spending aggregates against actual expenditures using a modern Bar graph.
- **Downloadable Reports:** One-click generation of beautifully formatted PDF financial reports including active ledger items, milestone progressions, and AI suggestions.
- **Global Currency Reactivity:** Live currency selector on the top navbar (INR `Rs. `, USD `\$`, EUR `EUR `, GBP `GBP `) which immediately re-calculates and re-formats every cash label in real-time.

[Tech Stack] Technology Stack

- **Frontend:** React.js, React Router DOM, Axios, Chart.js, React-Chartjs-2, Lucide React, Modern Custom CSS.

- **Backend:** Flask (Python), PyMongo, bcrypt, PyJWT, Python-Dotenv, Flask-CORS, OpenAI.
- **Database:** MongoDB (Atlas or Local fallback).

[Setup] Setup & Execution Guide

Prerequisites

Make sure the following are installed on your environment:

- [Node.js](https://nodejs.org/) (v16.0+)
- [Python](https://www.python.org/) (v3.8+)
- [MongoDB](https://www.mongodb.com/try/download/community) (Local server running on port `27017` or a MongoDB Atlas URI)

Step 1: Backend Setup (Flask Server)

1. Open your terminal, navigate to the `backend/` directory:

```
cd backend
```

2. Create a Python virtual environment and activate it:

- **Windows (PowerShell):**

```
python -m venv venv  
.\venv\Scripts\Activate.ps1
```

- **Mac/Linux:**

```
python3 -m venv venv  
source venv/bin/activate
```

3. Install the required dependencies:

```
pip install -r requirements.txt
```

4. Configure environment parameters in `backend/.env`:

- Open `backend/.env` in your text editor.
- Ensure `MONGO\_URI` is pointing to your active MongoDB instance.
- **(Optional)** Paste your OpenAI API Key into `OPENAI\_API\_KEY` to activate natural language coaching. If left blank, the app will run the statistical rules engine offline.

5. Start the Flask server:

```
python app/app.py
```

The server will initiate on `http://localhost:5000`.

Step 2: Frontend Setup (React App)

1. Open a new terminal window, navigate to the `frontend/` directory:

```
cd frontend
```

2. Ensure Vite dependencies are fully loaded:

```
npm install
```

3. Fire up the hot-reloading development server:

```
npm run dev
```

4. Open the displayed localhost URL in your browser:
Typically **`http://localhost:5173`**.

[Database collections] Database Collections Schema

`users`

- `\_id`: `ObjectId` (Primary Key)
- `username`: `String` (Unique)
- `email`: `String` (Unique)
- `password\_hash`: `String` (Securely salted and hashed)
- `created\_at`: `DateTime`

`expenses`

- `\_id`: `ObjectId` (Primary Key)
- `title`: `String`
- `amount`: `Double`
- `category`: `String` (Predefined validation set)
- `description`: `String` (Optional notes)
- `date`: `DateTime` (Parsed natively for fast range queries)
- `user\_id`: `ObjectId` (Foreign reference key)

`savings`

- `\_id`: `ObjectId` (Primary Key)
- `goal\_name`: `String`
- `target\_amount`: `Double`
- `saved\_amount`: `Double`
- `deadline`: `DateTime` (Goal deadline target)
- `user\_id`: `ObjectId` (Foreign reference key)

[Analytics] Analytical Reports & Printing

To export your records:

1. Navigate to the **AI Insights** page.
2. Review the charts, projections, and checkboxes checklist.
3. Click the **Export PDF Report** button at the bottom center.
4. This triggers a custom structured template containing print-optimized ledger lists, statistics summary blocks, and active target statuses, prompting the browser's native **Print to PDF** system. Ensure popups are allowed!